

Airbag ECU Portfolio HS Harz — Functional Safety

AIRBAG ELECTRONIC CONTROL UNIT

Measurement and Processing of Sensor Signals to Control Airbag Operation

HARDWARE ARCHITECTURE AND DESIGN SPECIFICATION (HADS)

Document Number	HADS-AIRBAG-ECU-001
Version	2.1
Date	2026-06-11
Status	RELEASED
Author	Om Naik, Mrunal Ghotekar
Applicable Standards	ISO 26262:2018 / IEC 61508:2010
ASIL Classification	ASIL D (ISO 26262) / SIL 3 (IEC 61508)
Target Hardware	Arduino Due (SAM3X8E) ×2, DM74LS08N, Waveshare SN65HVD230 ×2, USB-CAN-A
File	HADS-AIRBAG-ECU-001_Rev 2.1
Based On	HARA-AIRBAG-ECU-001 Rev 2.1 FSRs-AIRBAG-ECU-001 Rev 2.1 SC-AIRBAG-ECU-001 Rev 2.0

Table of Contents

1 Glossary	2
2 System Architecture Overview	2
2.1 Peer-Equal Design Principle	2
2.2 Deployment Decision Architecture	2
2.3 Node Inventory	3
3 Bill of Materials	3
4 Pin Assignments and Electrical Connections	4
4.1 Arduino Due — Pin Assignment (Identical for ECU 1 and ECU 2)	4
4.2 DM74LS08N AND Gate — Pin Assignment	5
4.3 Waveshare SN65HVD230 CAN Transceiver (Applies to Both Boards)	5
4.4 USB-CAN-A Adapter	5
5 Sensor Input Connections	5
5.1 Potentiometer Wiring	5
5.2 Baby Seat Switch	6
6 Output Connections	6
6.1 Airbag 1 LED — Driver Airbag Indicator (RED)	6
6.2 Airbag 2 LED — Passenger Airbag Indicator (RED)	6
6.3 Warning Lamp — DWI (RED LED, Shared)	7
7 CAN Bus Architecture	7
7.1 Physical Bus Configuration	7
7.2 CAN ID Assignment	7
7.3 CAN Frame Byte Layout	8
7.4 Status Byte Bit Fields (Byte 6)	8
8 Deployment Logic	8
8.1 Software Vote — Identical on Both ECUs	8
8.2 Communication Timeout Fault	9

8.3 Hardware AND Gate — 2oo2 Truth Table.....	9
9 Power Architecture	9
9.1 Power Rails	9
9.2 Voltage Domain Boundaries.....	9
10 List of Dependent Documents	10
11 Revision History	10
12 Approval and Sign-Off.....	10

1 Glossary

Table 1: Abbreviations and Definitions

Term	Definition
ADC	Analog-to-Digital Converter — converts 0–3.3 V analog input to 12-bit digital value 0–4095 on Arduino Due
ASIL	Automotive Safety Integrity Level — risk classification per ISO 26262 (QM, A, B, C, D)
CAN	Controller Area Network — serial communication bus
CRC	Cyclic Redundancy Check — frame integrity verification
DWI	Diagnostic Warning Indicator — shared red warning LED driven by both ECUs on pin D10
ECU	Electronic Control Unit
FSR	Functional Safety Requirement — from FSRs-AIRBAG-ECU-001
GPIO	General Purpose Input/Output — digital pin on microcontroller
HADS	Hardware Architecture and Design Specification — this document
HARA	Hazard and Risk Analysis
OMS	OMS Laboratory PC — passive CAN monitoring workstation running USBCANv2
POST	Power-On Self-Test — hardware verification sequence executed at startup
SIL	Safety Integrity Level per IEC 61508 (1–4)
VIH	Voltage Input High — minimum voltage logic gate recognises as logic HIGH
VOH	Voltage Output High — output voltage when gate output is logic HIGH
WDT	Hardware Watchdog Timer — SAM3X8E independent timer; resets MCU on timeout
2oo2	Two-out-of-Two — voting architecture requiring simultaneous agreement of both channels

2 System Architecture Overview

2.1 Peer-Equal Design Principle

Both ECUs are identical in hardware and nearly identical in firmware. The only differences between ECU 1 and ECU 2 are the three configuration defines at the top of the firmware:

Table 2: Firmware Configuration Differences

Constant	ECU 1 (Arduino Due 1)	ECU 2 (Arduino Due 2)
ECU_ID	1	2
DATA_ID (CAN TX ID)	0x100	0x200
REMOTE_ID (expected peer ID)	0x200	0x100

2.2 Deployment Decision Architecture

The system implements two independent AND conditions that must both be satisfied simultaneously before an airbag LED can activate:

- Software AND: voteDeploy = localCrash AND remoteCrash AND NOT systemFault AND NOT remoteFault. Each ECU independently confirms a crash using its own sensor readings before asserting its output pins HIGH.
- Hardware AND (DM74LS08N): the physical AND gate ensures the airbag LED cannot illuminate unless BOTH ECUs have simultaneously asserted their output pin HIGH.

This two-layer architecture ensures that a single software fault or single hardware fault in either channel cannot independently cause unintended deployment, satisfying FSR-03 and FSR-08 at ASIL D.

2.3 Node Inventory

Table 3: System Node Inventory

Node	Component	CAN TX ID	CAN RX ID	Role
ECU 1	Arduino Due 1	0x100	0x200	Peer — reads sensors, votes, transmits status
ECU 2	Arduino Due 2	0x200	0x100	Peer — reads sensors, votes, transmits status
OMS Lab PC	USBCANv2 software	—	0x100 and 0x200	Passive monitor — logs both channels
CAN Transceiver 1	Waveshare SN65HVD230	—	—	ECU 1 CAN0 to CANH/CANL bus
CAN Transceiver 2	Waveshare SN65HVD230	—	—	ECU 2 CAN0 to CANH/CANL bus
CAN Adapter	USB-CAN-A	—	—	CANH/CANL to USB — OMS PC interface

3 Bill of Materials

Table 4: Bill of Materials

Item	Description	Qty	Specification	Safety Role	FSR
Arduino Due	SAM3X8E ARM Cortex-M3 84 MHz microcontroller	2	3.3 V logic, 12-bit ADC, CAN0 hardware	Processing channel — ASIL D	FSR-07
Potentiometer 10 kΩ	Rotary potentiometer — sensor simulation	3	Linear taper, 10 kΩ, 3-terminal	Accelerometer / gyroscope / pressure simulation	FSR-01
Tactile push button	Momentary push button — baby seat switch	1	Active-LOW, SPST, normally open	Baby seat occupant detection — ASIL D	FSR-09
DM74LS08N	Quad 2-input AND gate IC, 14-pin DIP	1	74LS TTL, VCC 5 V, VIH min 2.0 V	Hardware 2oo2 physical deployment layer — ASIL D	FSR-03
Waveshare SN65HVD230	CAN transceiver board	2	3.3 V supply, 500 kbit/s, CANH/CANL output	CAN bus physical layer — ASIL B	FSR-13
USB-CAN-A	USB to CAN bus adapter	1	500 kbit/s, USBCANv2 compatible	OMS PC CAN monitoring interface	FSR-16
Resistor 10 kΩ (pull-up)	Baby seat switch pull-up on D11	1	1/4 W — connected to Arduino Due 3.3 V pin	Safe default HIGH state for D11 — ASIL D	FSR-10
Resistor 220 Ω — airbag 1	LED current limiter for airbag 1 indicator	1	1/4 W	Limits current through airbag 1 LED	—
Resistor 220 Ω — airbag 2	LED current limiter for airbag 2 indicator	1	1/4 W	Limits current through airbag 2 LED	—

Resistor 220 Ω — DWI ($\times 2$)	LED current limiter for warning lamp	2	1/4 W — one per ECU D10 output	Limits current through DWI LED	FSR-19
LED red — airbag 1	Driver airbag deployment indicator	1	3 mm RED, Vf ~2.0 V	Simulates driver airbag deployment	—
LED red — airbag 2	Passenger airbag deployment indicator	1	3 mm RED, Vf ~2.0 V	Simulates passenger airbag deployment	—
LED red — DWI	Diagnostic Warning Indicator lamp	1	3 mm RED, Vf ~2.0 V — shared anode	Warning lamp driven by both ECU D10 pins	FSR-19
Breadboard	Full-size 830 tie-point solderless	1	Standard 0.1 inch pitch	All passive and sensor wiring	—
Jumper wires M-M	Dupont jumper wire set	1 set	Various lengths	All breadboard interconnections	—
Twisted pair wire	CANH/CANL bus wiring	30 cm	Min. 1 twist per cm	CAN bus noise immunity	FSR-13

4 Pin Assignments and Electrical Connections

4.1 Arduino Due — Pin Assignment (Identical for ECU 1 and ECU 2)

Table 5: Arduino Due Pin Assignment

Pin	Signal	Connected To	Dir	Notes
A0	Accelerometer POTI-1	Potentiometer 1 wiper (leg 3)	In	0–3.3 V analog, 12-bit ADC. Signal line shared with peer ECU — high-impedance input, safe to share
A1	Gyroscope POTI-2	Potentiometer 2 wiper (leg 3)	In	0–3.3 V analog, 12-bit ADC. Shared
A2	Pressure POTI-3	Potentiometer 3 wiper (leg 3)	In	0–3.3 V analog, 12-bit ADC. Shared
D8	Airbag 1 AND gate vote	DM74LS08N pin 1 (ECU 1) / pin 2 (ECU 2)	Out	HIGH = vote to deploy airbag 1; LOW = inhibit
D9	Airbag 2 AND gate vote	DM74LS08N pin 4 (ECU 1) / pin 5 (ECU 2)	Out	HIGH = vote to deploy airbag 2; LOW = inhibit or baby seat suppression active
D10	DWI warning lamp	220 Ω → DWI LED anode (shared)	Out	HIGH = system fault active. Both ECU D10 outputs drive the same LED — hardware OR logic
D11	Baby seat switch	Push button + 10 k Ω pull-up to Arduino Due 3.3 V pin	In	Active-LOW. LOW = baby seat present; HIGH = adult / empty. External pull-up to 3.3 V
D13	Status LED	Built-in LED on Arduino Due board	Out	Blinks at 500 ms — indicates main loop is running
CAN TX	CAN transmit	Waveshare SN65HVD230 CANTX pin	Out	CAN0 hardware controller. ECU 1 TX ID 0x100; ECU 2 TX ID 0x200
CAN RX	CAN receive	Waveshare SN65HVD230 CANRX pin	In	ECU 1 filters for 0x200; ECU 2 filters for 0x100
3.3 V	3.3 V supply output	Waveshare VCC pin + D11 pull-up resistor	Pwr	Supplies CAN transceiver and pull-up. Must not exceed 3.3 V load capacity
5 V	5 V supply output	Common 5 V breadboard rail	Pwr	Supplies DM74LS08N VCC (pin 14) only
GND	Ground	Common GND breadboard rail + Waveshare GND	Pwr	Single common ground reference for all components

4.2 DM74LS08N AND Gate — Pin Assignment

Table 6: DM74LS08N Pin Assignment

Pin	Name	Connected To	Function
1	1A	ECU 1 (Due 1) D8	Airbag 1 deploy vote — Channel A input
2	1B	ECU 2 (Due 2) D8	Airbag 1 deploy vote — Channel B input
3	1Y	220 Ω → Airbag 1 LED anode	Gate 1 output — HIGH only when pin 1 AND pin 2 both HIGH
4	2A	ECU 1 (Due 1) D9	Airbag 2 deploy vote — Channel A input
5	2B	ECU 2 (Due 2) D9	Airbag 2 deploy vote — Channel B input
6	2Y	220 Ω → Airbag 2 LED anode	Gate 2 output — HIGH only when pin 4 AND pin 5 both HIGH
7	GND	Common GND breadboard rail	IC ground reference
8–13	n/c	Unconnected	Gates 3 and 4 unused — leave floating
14	VCC	Common 5 V breadboard rail	IC power supply — 5 V

4.3 Waveshare SN65HVD230 CAN Transceiver (Applies to Both Boards)

Table 7: Waveshare SN65HVD230 Pin Connections

Pin	Connected To	Notes
CANTX	Arduino Due CAN TX pin	Logic-level transmit input from CAN0 controller
CANRX	Arduino Due CAN RX pin	Logic-level receive output to CAN0 controller
3.3 V	Arduino Due 3.3 V pin	VCC — must be 3.3 V. Connecting to 5 V will destroy the IC
GND	Common GND breadboard rail	Signal reference — must be common with Arduino Due GND
CANH	Common CANH bus rail	Differential CAN high — shared by both transceivers and USB-CAN-A
CANL	Common CANL bus rail	Differential CAN low — shared by both transceivers and USB-CAN-A

4.4 USB-CAN-A Adapter

Table 8: USB-CAN-A Connections

Pin	Connected To	Notes
CANH	Common CANH bus rail	Connects to both Waveshare CANH pins on the shared bus
CANL	Common CANL bus rail	Connects to both Waveshare CANL pins on the shared bus
USB	OMS Lab PC USB port	Configure USBCANV2 to 500 kbit/s — monitors both CAN IDs (0x100 and 0x200)

5 Sensor Input Connections

5.1 Potentiometer Wiring

Both ECUs are identical in hardware and nearly identical in firmware. Neither ECU is a master or slave. The only differences between ECU 1 and ECU 2 are the three configuration defines at the top of the firmware:

```

accel = (rawA0 / 4095.0) * 100.0    // 0-100 g           threshold: 30.0 g
gyro  = (rawA1 / 4095.0) * 360.0    // 0-360 deg/s      threshold: 120.0 deg/s
press = (rawA2 / 4095.0) * 500.0    // 0-500 kPa        threshold: 200.0 kPa

```

Table 9: Potentiometer Terminal Connections

Terminal	Connected To	Notes
----------	--------------	-------

Leg 1 (supply end)	Common 3.3 V breadboard rail	Top of resistive track. Use 3.3 V to constrain wiper to safe ADC range
Leg 2 (GND end)	Common GND breadboard rail	Bottom of resistive track
Leg 3 (wiper)	ADC pin of both Due 1 and Due 2 (shared wire)	Output voltage 0–3.3 V. Both ADC inputs are high-impedance — safe to share

Table 10: Potentiometer Sensor Mapping

Potentiometer	Due 1 Pin	Due 2 Pin	Physical Range	Deploy Threshold	ADC Threshold (approx)
POTI-1 — Accelerometer	A0	A0	0–100 g	30.0 g	raw > 1228
POTI-2 — Gyroscope	A1	A1	0–360 deg/s	120.0 deg/s	raw > 1365
POTI-3 — Pressure	A2	A2	0–500 kPa	200.0 kPa	raw > 1638

5.2 Baby Seat Switch

Firmware reads: `babySeat = (digitalRead(PIN_BABY) == LOW)`. Pin D11 configured as INPUT — external pull-up provided on hardware.

Table 11: Baby Seat Switch Connections

Connection	Detail
Switch terminal 1	Common GND rail — pressing the button pulls D11 to 0 V
Switch terminal 2	D11 of Due 1 AND D11 of Due 2 (shared) AND one end of 10 k Ω pull-up resistor
Pull-up resistor	10 k Ω — one end to D11 signal line, other end to Arduino Due 3.3 V pin
Logic HIGH — button open	~3.3 V via pull-up — no baby seat detected / adult occupant — passenger airbag armed
Logic LOW — button pressed	~0 V — baby seat present — D9 held LOW — passenger airbag suppressed

6 Output Connections

All LEDs are 3 mm red. The DM74LS08N AND gate outputs (pins 3 and 6) drive the airbag LEDs directly through 220 Ω current-limiting resistors — confirmed operating correctly in the laboratory with 3 mm red LEDs.

6.1 Airbag 1 LED — Driver Airbag Indicator (RED)

Table 12: Airbag 1 LED Circuit

Connection	Detail
AND gate pin 3 (1Y)	→ 220 Ω resistor → Airbag 1 LED positive (anode)
LED positive (anode)	→ 220 Ω resistor → Airbag 1 LED positive (anode)
LED cathode	→ Common GND rail
LED cathode	→ Common GND rail
LED colour	RED — V_f ~2.0 V
LED current	3 mm red LED, V_f ~2.0 V, direct drive from AND gate output
Active condition	AND gate pin 3 HIGH — requires ECU1 D8 HIGH AND ECU2 D8 HIGH AND voteDeploy TRUE on both

6.2 Airbag 2 LED — Passenger Airbag Indicator (RED)

Same circuit as Airbag 1. AND gate pin 6 drives the LED directly through a 220 Ω resistor.

Table 13: Airbag 2 LED Circuit

Connection	Detail
AND gate pin 6 (2Y)	→ 220 Ω resistor → Airbag 2 LED positive (anode)
LED positive (anode)	→ 220 Ω resistor → Airbag 2 LED positive (anode)
LED cathode	→ Common GND rail
LED colour	RED — 3 mm red LED, same type as Airbag 1
Active condition	AND gate pin 6 HIGH — requires ECU1 D9 HIGH AND ECU2 D9 HIGH. D9 is only HIGH when voteDeploy=TRUE AND babySeat=FALSE

6.3 Warning Lamp — DWI (RED LED, Shared)

Both ECU D10 pins connect to the same LED through separate 220 Ω resistors. Either pin driving HIGH illuminates the lamp — OR gate behaviour implemented in hardware.

Table 14: DWI Warning Lamp Circuit

Connection	Detail
ECU 1 D10	→ 220 Ω resistor → DWI LED positive (anode)
ECU 2 D10	→ 220 Ω resistor → same DWI LED positive (anode)
LED cathode	→ Common GND rail
LED colour	RED — 3 mm red LED, same type as airbag LEDs
Active condition	Either ECU detects systemFault = TRUE (CPU / RAM / ADC / COMM fault)
FSR	FSR-19 — DWI must activate within 800 ms of fault detection

7 CAN Bus Architecture

7.1 Physical Bus Configuration

Table 15: CAN Bus Physical Parameters

Parameter	Value
Bus topology	Single segment — three nodes on one twisted pair cable
Baud rate	500 kbit/s — confirmed in firmware: Can0.begin(CAN_BPS_500K)
Termination	120 Ω at each physical end — two resistors total
Cable type	Twisted pair — minimum 1 twist per cm
Bus nodes	Waveshare 1 (ECU 1), Waveshare 2 (ECU 2), USB-CAN-A (OMS PC)
Terminator — position A	At the Waveshare 1 / ECU 1 end of the bus
Terminator — position B	At the USB-CAN-A / OMS PC end of the bus
Waveshare 2	Middle node — no termination resistor on this board

7.2 CAN ID Assignment

Table 16: CAN ID Assignment

Board	TX ID	RX Filter	Frame Rate	Note
Arduino Due 1 (ECU 1)	0x100	Can0.watchFor() — software filters for incoming.id == 0x200	~500 ms	Heartbeat interval HEARTBEAT_MS = 500

Arduino Due 2 (ECU 2)	0x200	Can0.watchFor() — software filters for incoming.id == 0x100	~500 ms	Identical logic — filters for 0x100 in receiveCAN()
OMS PC (USBCANV2)	—	Monitors 0x100 and 0x200	Passive	Configure USBCANV2 to 500 kbit/s

7.3 CAN Frame Byte Layout

Table 17: CAN Frame Byte Layout (DLC = 8)

Byte	Field	Encoding	Firmware Reference
0	A0 raw — low byte	rawA0 & 0xFF	frame.data.bytes[0] = a0 & 0xFF
1	A0 raw — high byte	rawA0 >> 8	frame.data.bytes[1] = a0 >> 8
2	A1 raw — low byte	rawA1 & 0xFF	frame.data.bytes[2] = a1 & 0xFF
3	A1 raw — high byte	rawA1 >> 8	frame.data.bytes[3] = a1 >> 8
4	A2 raw — low byte	rawA2 & 0xFF	frame.data.bytes[4] = a2 & 0xFF
5	A2 raw — high byte	rawA2 >> 8	frame.data.bytes[5] = a2 >> 8
6	Status byte	Bit 0 = crashLocal, Bit 1 = babySeat, Bit 2 = systemFault, Bits 3–7 = 0	frame.data.bytes[6] = status
7	ECU ID	1 (ECU 1) or 2 (ECU 2)	frame.data.bytes[7] = ECU_ID

7.4 Status Byte Bit Fields (Byte 6)

Table 18: Status Byte Bit Fields

Bit	Field	Value 1 Meaning	Value 0 Meaning
Bit 0	localCrash	Crash detected — at least one sensor exceeded deployment threshold on this ECU	No crash detected
Bit 1	babySeat	Baby seat present — D11 LOW — passenger airbag suppressed	No baby seat / adult occupant
Bit 2	systemFault	Fault active — fault_cpu, fault_ram, fault_adc, or fault_comm is TRUE	No fault
Bits 3–7	Reserved	0	Unused — reserved for future use

8 Deployment Logic

8.1 Software Vote — Identical on Both ECUs

The following deployment vote logic is implemented identically on both ECUs.

```
localCrash   = (accel > 30.0) || (gyro > 120.0) || (press > 200.0)
systemFault  = faultCPU || faultRAM || faultADC || faultCOMM
voteDeploy   = localCrash && remoteCrash && !systemFault && !remoteFault
```

```
D8 = voteDeploy           // Airbag 1 AND gate input
D9 = voteDeploy && !babySeat // Airbag 2 AND gate input — passenger suppression
D10 = systemFault         // DWI warning lamp
```

remoteCrash and remoteFault are extracted from the received CAN frame byte 6 of the other ECU:

```
remoteCrash = (status & 0x01) // bit 0 of received byte 6
```



```
remoteFault = (status & 0x04) // bit 2 of received byte 6
```

8.2 Communication Timeout Fault

Each ECU monitors the timestamp of the last received peer frame:

```
if (millis() - lastRemoteHB > COMM_TIMEOUT) faultCOMM = true
```

If no valid peer frame is received within COMM_TIMEOUT, faultCOMM is set TRUE. This propagates to systemFault = TRUE, voteDeploy = FALSE, and D8/D9 driven LOW. The AND gate output drops LOW and no airbag LED fires. This is the fail-safe state on communication loss.

8.3 Hardware AND Gate — 2oo2 Truth Table

Table 19: AND Gate 2oo2 Deployment Truth Table

ECU 1 D8	ECU 2 D8	AND Gate Pin 3 Output	Airbag 1 LED
LOW	LOW	LOW	OFF — no crash confirmed on either ECU
HIGH	LOW	LOW	OFF — ECU 2 not confirming crash
LOW	HIGH	LOW	OFF — ECU 1 not confirming crash
HIGH	HIGH	HIGH	FIRE — both ECUs simultaneously confirm crash

The same truth table applies to D9 / AND gate pin 6 / Airbag 2 LED, with the additional condition that D9 is only HIGH when voteDeploy=TRUE AND babySeat=FALSE.

9 Power Architecture

9.1 Power Rails

Table 20: Power Rail Summary

Rail	Source	Voltage	Consumers	Notes
5 V	Arduino Due 5 V pin (USB-powered)	5 V	DM74LS08N VCC (pin 14) only	All other logic operates at 3.3 V. Do not use this rail for any signal-level pull-up
3.3 V — ECU 1	Arduino Due 1 3.3 V pin	3.3 V	Waveshare 1 VCC, 10 kΩ baby seat pull-up resistor	Powers CAN transceiver 1 and D11 pull-up
3.3 V — ECU 2	Arduino Due 2 3.3 V pin	3.3 V	Waveshare 2 VCC	Powers CAN transceiver 2
GND	Common breadboard GND rail	0 V	All components	Single ground reference — mandatory for signal integrity across all nodes

9.2 Voltage Domain Boundaries

Table 21: Voltage Domain Interface Summary

Interface	Specification
Potentiometer leg 1 supply	3.3 V rail — constrains wiper output to 0–3.3 V, within safe ADC input range
Baby seat pull-up (D11)	10 kΩ to Arduino Due 3.3 V pin — D11 reads 3.3 V (open) and 0 V (pressed) — within GPIO specification
Due D8/D9 (3.3 V) → DM74LS08N inputs	74LS VIH minimum = 2.0 V — 3.3 V logic level is sufficient to guarantee HIGH recognition
DM74LS08N outputs → airbag LEDs	VOH ~2.7 V — 3 mm red LED Vf ~2.0 V — direct drive through 220 Ω confirmed operating correctly

Waveshare SN65HVD230
VCC

Must be 3.3 V — supplied from Arduino Due 3.3 V pin. Connecting to 5 V will destroy the IC

10 List of Dependent Documents

Table 22: Dependent Documents

No.	Document Name	Document Number	Revision	Date
1	Hazard and Risk Analysis	HARA-AIRBAG-ECU-001	Rev 2.1	2026-05-06
2	Functional Safety Requirements Specification	FSRS-AIRBAG-ECU-001	Rev 2.1	2026-05-11
3	Safety Concept Document	SC-AIRBAG-ECU-001	Rev 2.0	2026-05-12
5	ISO 26262:2018 — Road Vehicles — Functional Safety	—	2018 Edition	—
6	IEC 61508:2010 — Functional Safety of E/E/PE Systems	—	2010 Edition	—

11 Revision History

Table 23: Revision History

Rev	Date	Author	Description
1.0	2026-05-31	Hardware Safety Engineering Team	Initial release. Full HADS covering breadboard wiring, BOM, pin assignments, power architecture, CAN bus, and safety review.
2.0	2026-06-10	Hardware Safety Engineering Team	Peer-equal architecture confirmed. Firmware deployment logic, CAN frame layout, and timeout handling documented. All LEDs confirmed 3 mm red. Direct LED drive from AND gate confirmed correct. Baby seat pull-up moved to 3.3 V.
2.1	2026-06-11	Hardware Safety Engineering Team	Final clean release. Removed version-change notes, closed-issue histories, and superseded content. Potentiometer leg 1 confirmed on 3.3 V rail. All voltage domain interfaces confirmed correct.

12 Approval and Sign-Off

Table 24: Approval and Sign-Off

Role	Name	Signature	Date
Hardware Safety Engineer			
Software Safety Engineer			
Functional Safety Engineer			
V&V Lead			
Project Supervisor			